DAO

Desarrollo de Aplicaciones con Objetos

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

{ }

```
class DAO:
    def aprender():
        return software
```

Plantilla institucional de clases

Versión 2026

DAO

Plantel Docente

```python
class Alumno:
    def __init__(self, legajo,
nombre):
        self.legajo = legajo
        self.nombre = nombre

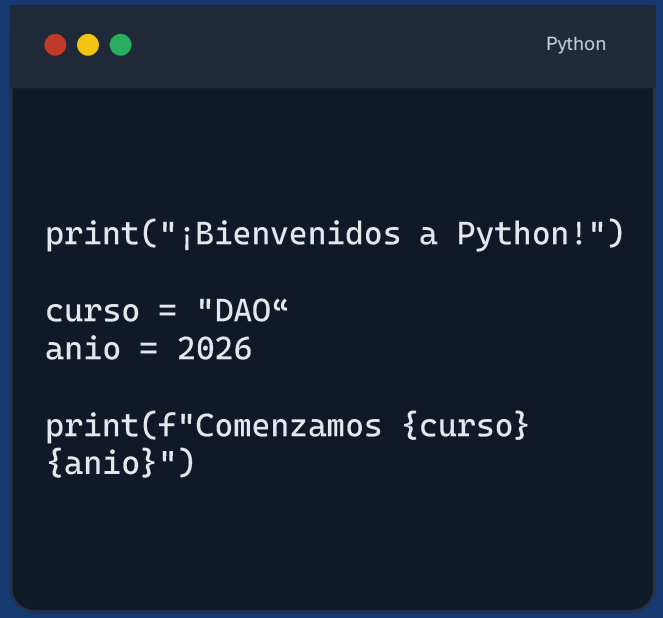
    def presentarse(self):
        return self.nombre
```

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

DAO

Temas Clase

- Uso de archivos plano
- Lectura y manipulacion de archivos txt y csv
- Lectura y manipulacion de archivos json
- Conjuntos
- Diccionarios



DAO

Material Disponible

Aula Virtual: DAO 2026

Apunte: Material de la Cátedra

Videos – Tutoriales

Playlist https://www.youtube.com/@DAO-UTN

Lectura de Archivos

https://www.youtube.com/watch?v=im8BCHSyrFE

Lectura de Archivos TXT

https://www.youtube.com/watch?v=Fxvo18vB0SU

Lectura de Archivos CSV

https://www.youtube.com/watch?v=zRlxad7d5KY



Archivos

Es una estructura de datos lineal almacenada en dispositivos de almacenamiento externos (como discos o memorias flash).

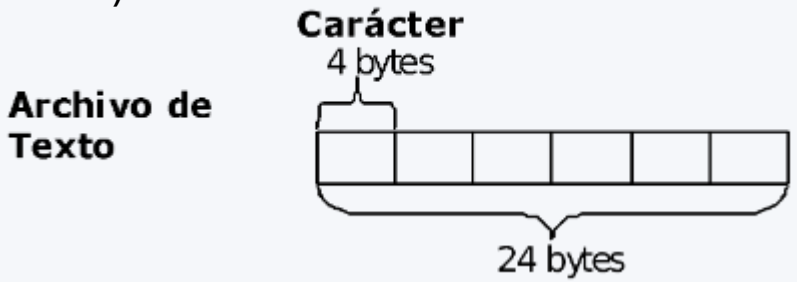
Los datos que se graban en un archivo se representan en sistema binario y por cada dato se utilizan tantos bytes como sea necesario para representar ese dato.

El *tamaño de un archivo* es la *cantidad total de bytes* que contiene el archivo.

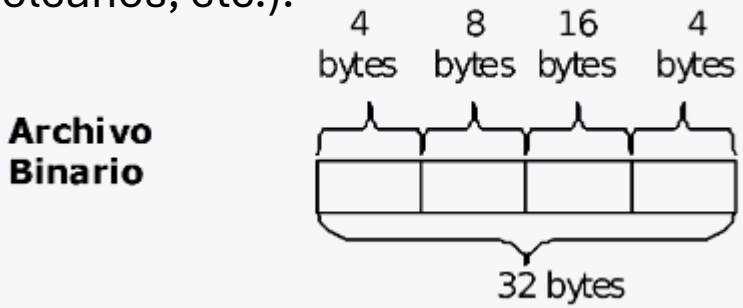


Tipo de Archivos

Archivos de texto: Todos los bytes del archivo son interpretados como caracteres (en base, por ejemplo, a la tabla ASCII).



Archivos binarios: Las secuencias o bloques de bytes que el archivo contiene representan información de cualquier tipo (números en formato binario, caracteres, valores booleanos, etc.).





Archivos de Texto

1 Formato

- El lenguaje Python emplea por default el estándar Unicode bajo la forma de codificación UTF-8 (Unicode Trasformation Format o Formato de Transformación Unicode) para interpretación de texto.
- Como el estándar ASCII está incluido como un subconjunto de Unicode, cualquier archivo de texto con formato ASCII puro será leído y gestionado sin problemas.
- Cuando se abre un archivo de texto en Python, se pueden leer y grabar cadenas de caracteres en ese archivo con relativa sencillez.

2 Apertura

La función open() es la encargada de abrir el canal de comunicación entre el dispositivo que contiene al archivo y la memoria principal.

Definición general:
open (nombre físico o file descriptor, modo de apertura)
Ejemplos:

```
m = open("datos.txt", "w")
m = open("c:\\documents\\datos.txt", "w")
```

3 Cierre

- La función close() cierra el canal de comunicación, cierra la conexión y libera cualquier recurso.
- Luego de invocar a close(), la variable u objeto que representaba al archivo queda indefinida.
- En Python los archivos son cerrados automáticamente cuando la variable para accederlos sale del ámbito.
- El programador debe estar seguro que el archivo que estuvo manejando se cierre oportunamente, ya sea en forma manual o en forma automática.

Archivos de Texto: Leer

Para leer el archivo completo, línea por línea, se puede usar un ciclo for que itere sobre el contenido del file object:

```
def leer_con_iterador():
    m = open('datos.txt', 'r')
    print('\nContenido con iterador:')
    for line in m:
        print(line, end='')
    m.close()
```

readlines(): Retorna una lista (un arreglo) que contiene a todas las líneas del archivo .

readlines(n) Intentará recuperar hasta n bytes del archivo. En este caso, solo incluirá en la lista retornada las cadenas completas que haya logrado leer:

```
def list_lines():
    m = open('datos.txt', 'r')
    lista = m.readlines()
    print('\nLista de lineas
contenidas:')
    print(lista)
    m.close()
```

Uso de With:

```
with open("mi_archivo.txt", "w") as archivo:
  archivo.write("Esta línea se escribe usando el
bloque with.\n")
```

Archivos de Texto: Escribir

Ejemplo Escribir

```
# Abre (o crea) un archivo en modo de escritura
archivo = open("mi_archivo.txt", "w")
# Escribe una línea de texto en el archivo
archivo.write("¡Hola, este es el contenido de mi archivo!\n")
# Escribe otra línea de texto en el archivo
archivo.write("Aquí va otra línea de texto.\n")
# Cierra el archivo
archivo.close()
```

Ejemplos Agregar

```
# Abre el archivo en modo de anexado (append)
archivo = open("mi_archivo.txt", "a")
# Escribe otra línea de texto al final del archivo
archivo.write("Esta línea se añadió al final del archivo.\n")
# Cierra el archivo
archivo.close()
```

Archivos de Texto: Metodos Comunes

Método	Aplicación
closed()	Retorna True si el archivo está cerrado.
flush()	Vuelca al archivo los buffers de grabación, si corresponde. Aplicable sólo archivos abiertos para grabar (no hace nada en caso contrario).
readable()	Retorna True si el archivo está disponible para ser leído.
truncate(size=None)	Trunca el contenido del archivo a una cantidad igual a size bytes. El archivo puede aumentar o disminuir su tamaño.
writable()	Retorna True si el archivo está disponible para ser grabado.
writelines(lines)	Graba el contenido de la lista lines en el archivo, por defecto sin incluir separadores de línea (por lo cual, el programador debe indicar ese separador al final de cada elemento de la lista lines si quiere los separadores).

Archivos JSON

- JSON (JavaScript Object Notation) es un formato de intercambio de datos que se utiliza para representar información estructurada de manera legible tanto para humanos como para máquinas.
- Diseñado originalmente como un subconjunto del lenguaje JavaScript, actualmente es ampliamente utilizado en una variedad de lenguajes de programación.
- Los archivos JSON están compuestos por una colección de pares clave-valor, donde las claves son cadenas de texto y los valores pueden ser cadenas, números, booleanos, objetos, matrices u otros valores JSON.

```
{
 "universidad": "Universidad Tecnológica Nacional",
 "regional": "Regional Córdoba",
 "materia": "DAO",
 "año": 2026,
 “contenido”:[“Python”,”Objetos”,”Code”]
}
```

Archivos de Texto

1 Formato

En Python, se puede utilizar la librería incorporada llamada json para leer e interpretar archivos JSON de manera sencilla.

Supongamos que hay un archivo llamado "datos.json" con el siguiente contenido:

```
{
  "nombre": "Juan",
  "edad": 30,
  "soltero": true,
  "pasatiempos": ["fútbol", "lectura", "viajes"]
}
```

2 Lectura

```
import json

# Abre el archivo JSON en modo lectura
with open('datos.json', 'r') as archivo:
    datos = json.load(archivo)

# Ahora, 'datos' es un diccionario que contiene la información del archivo JSON
print(datos['nombre'])  # Imprime: Juan
print(datos['edad'])    # Imprime: 30
print(datos['pasatiempos'])  # Imprime: ['fútbol', 'lectura', 'viajes']
```

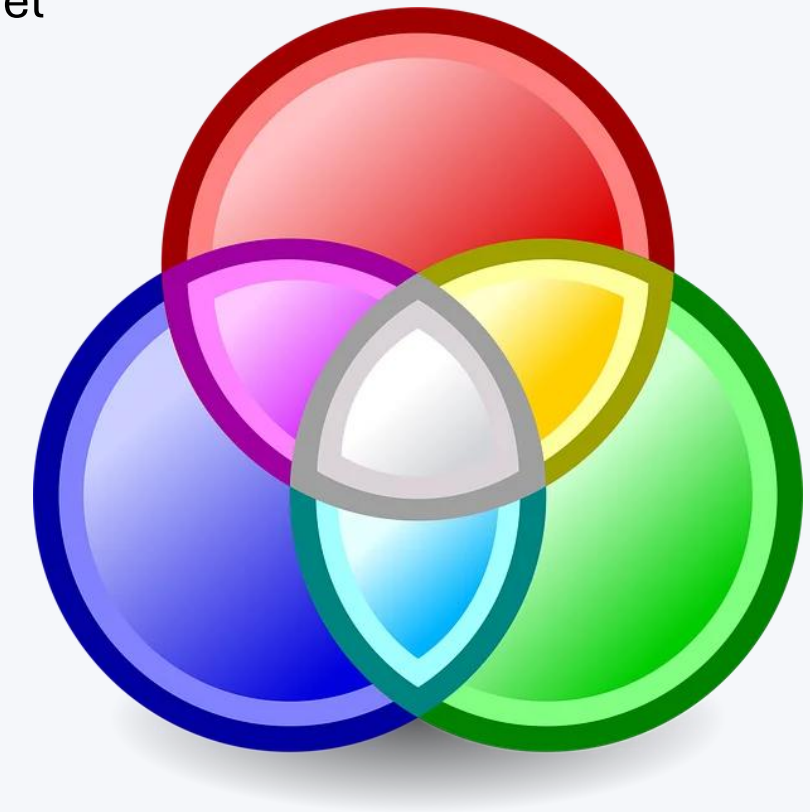
3 Escritura

```
import json
datos= {
    'nombre':'Juan Perez',
    'edad':30,
    'profesion':'estudiante'
    }
# Abre el archivo JSON en modo escritura
nombre_archivo = 'datos_simples.json'
# Grabar el archivo JSON
with open(nombre_archivo, 'w') as archivo_json:
    json.dump(datos, archivo_json, indent=4)


print(f'Datos grabados en {nombre_archivo}')
```

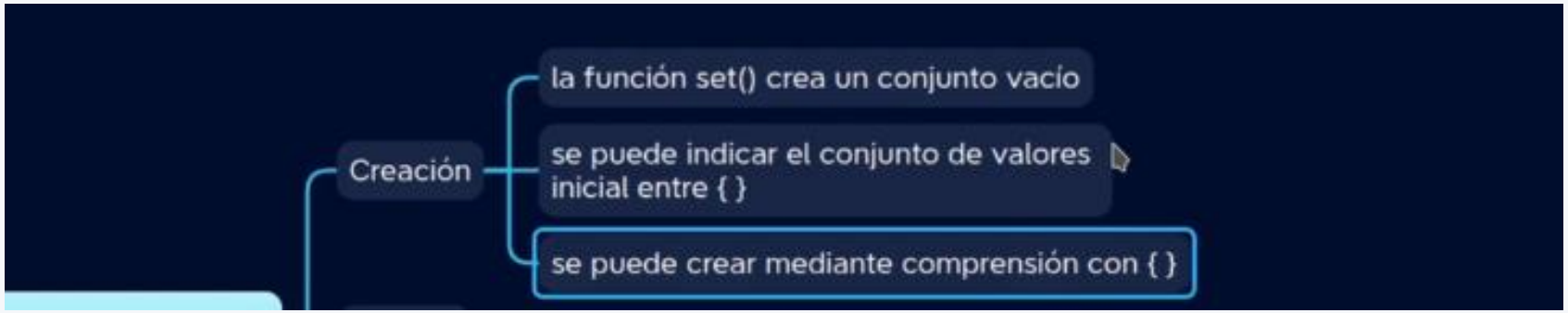
Conjuntos

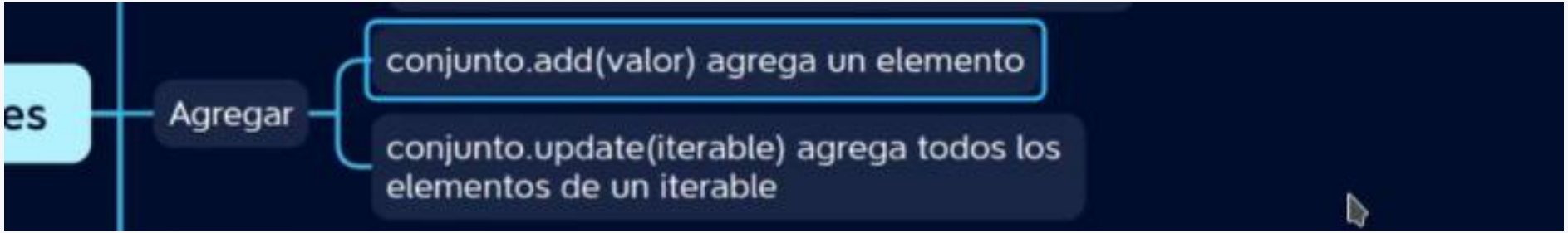
- Los conjuntos son una estructura de datos proporcionada por el lenguaje que permite almacenar y manipular colecciones de elementos.
- A diferencia de las listas o las tuplas, los conjuntos tienen características distintivas que los hacen únicos:
- Elementos únicos
- No ordenados
- Mutables (pueden cambiar luego de creados)
- Admiten operaciones de conjuntos:
  - union()
  - intersection()
  - difference()
- Heterogéneos (elementos de distintos tipos)



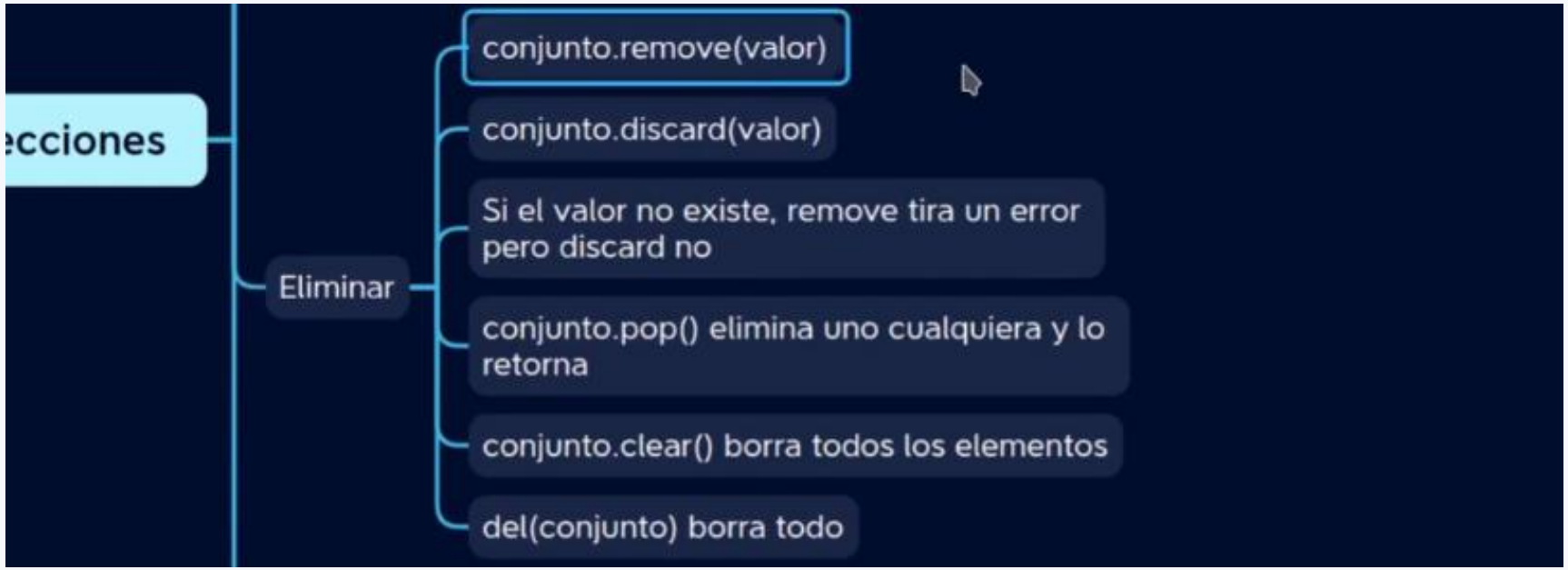
Conjuntos

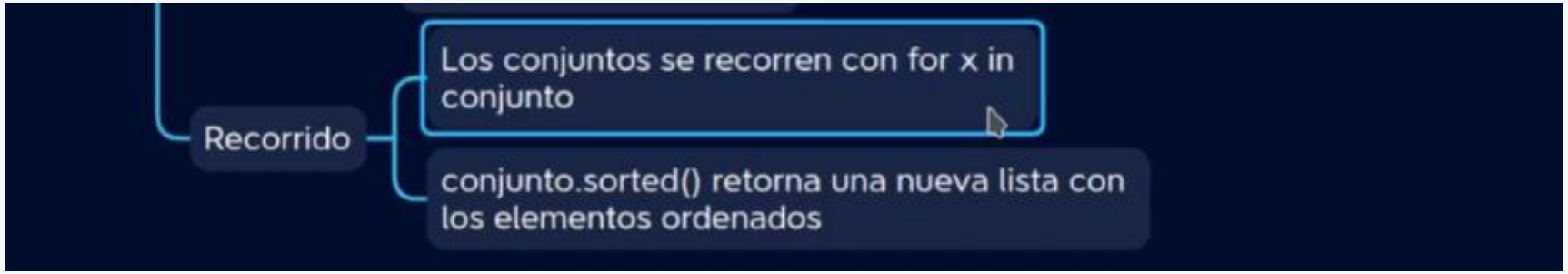

Características
Coleccion de datos
No aceptan repetidos
No tienen índices
Las búsquedas son muy rápidas


Creación
la función set() crea un conjunto vacío
se puede indicar el conjunto de valores inicial entre { }
se puede crear mediante comprensión con { }

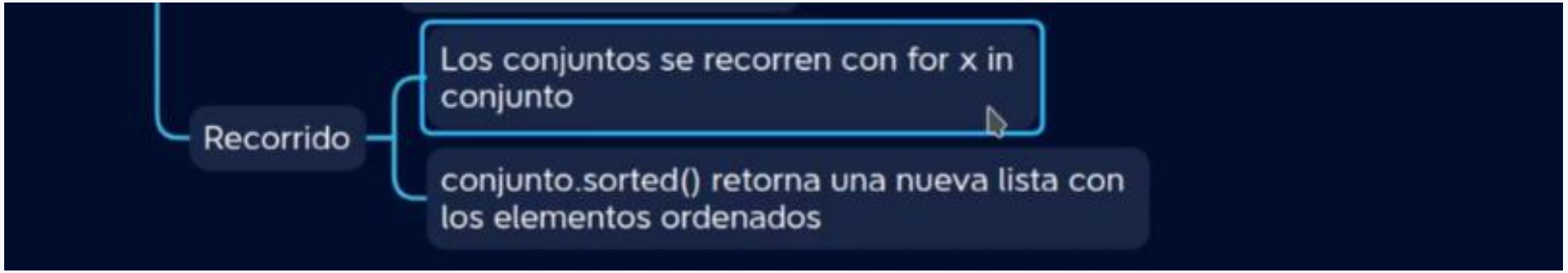

es
Agregar
conjunto.add(valor) agrega un elemento
conjunto.update(iterable) agrega todos los elementos de un iterable

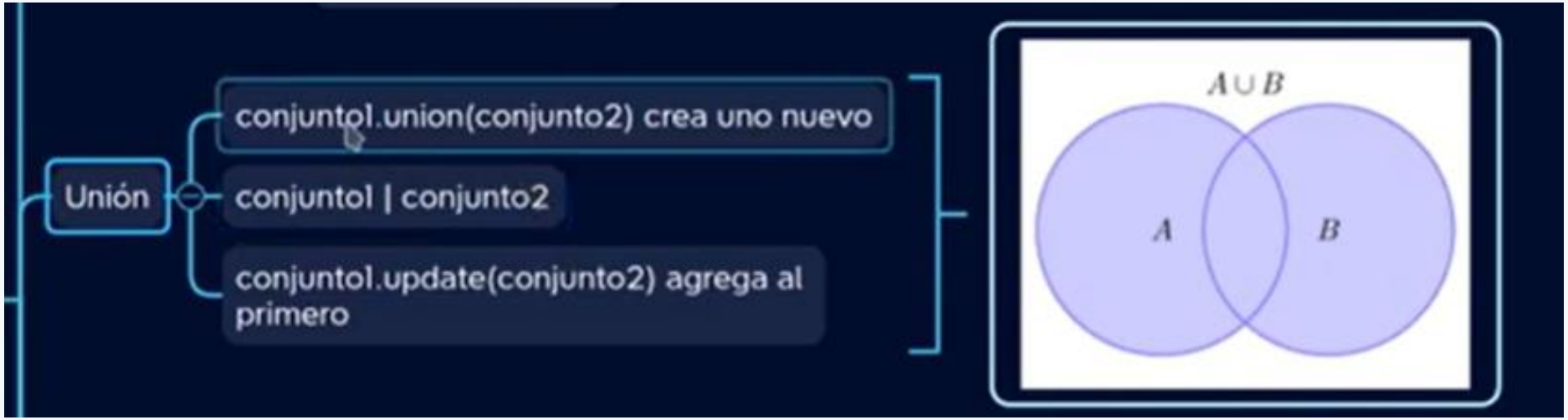
Conjuntos


ecciones
Eliminar
conjunto.remove(valor)
conjunto.discard(valor)
Si el valor no existe, remove tira un error pero discard no
conjunto.pop() elimina uno cualquiera y lo retorna
conjunto.clear() borra todos los elementos
del(conjunto) borra todo

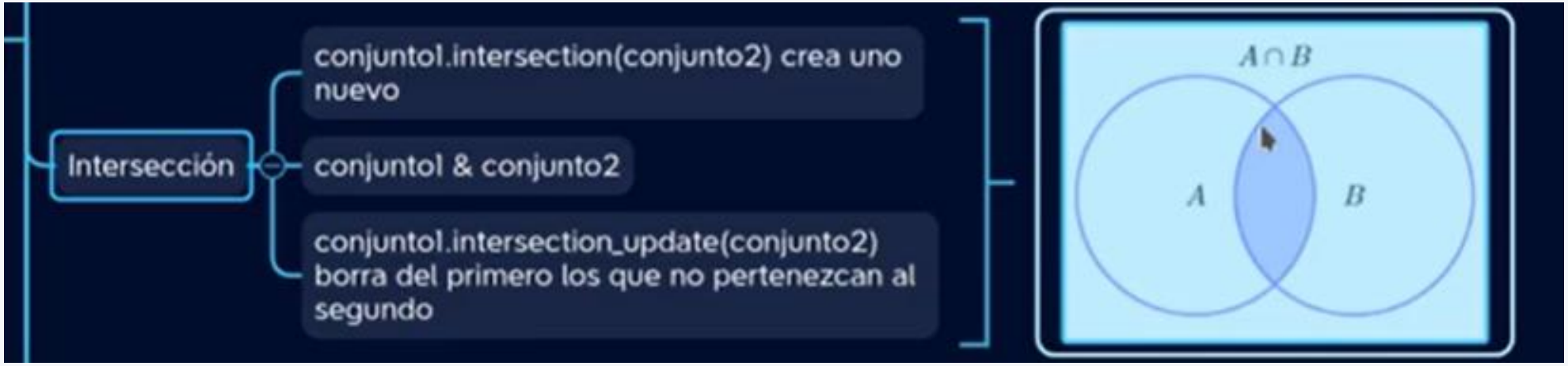

Recorrido
Los conjuntos se recorren con for x in conjunto
conjunto.sorted() retorna una nueva lista con los elementos ordenados

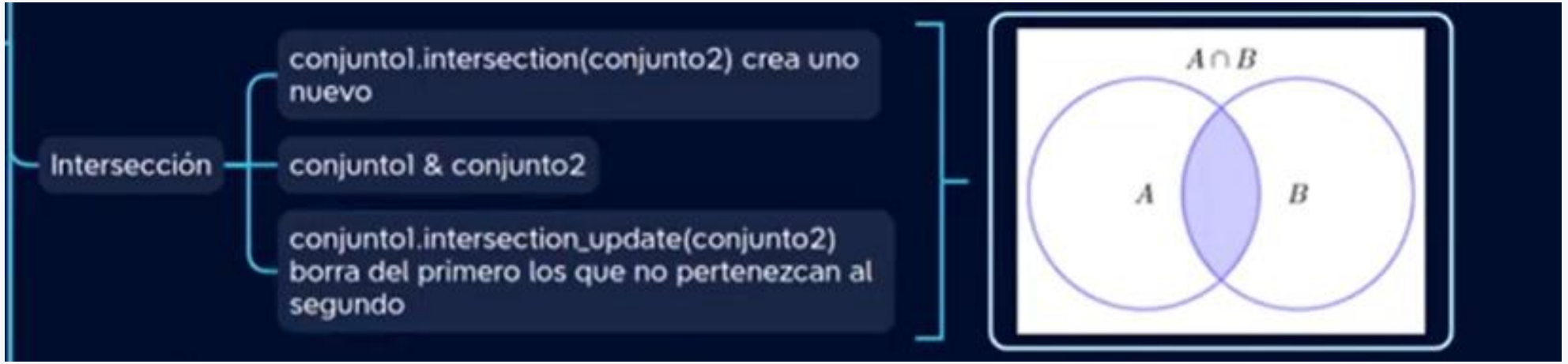
Conjuntos – Operaciones



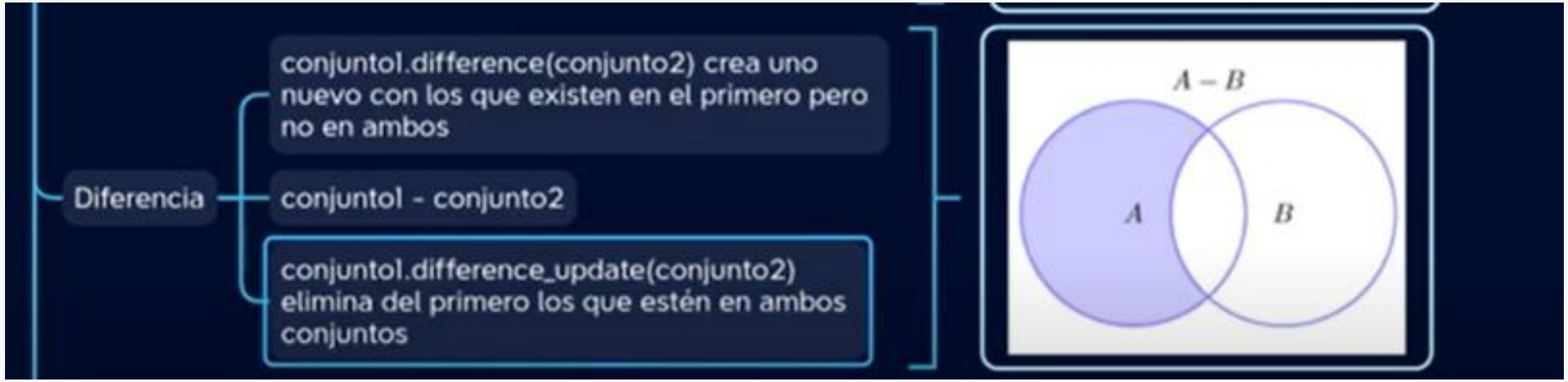


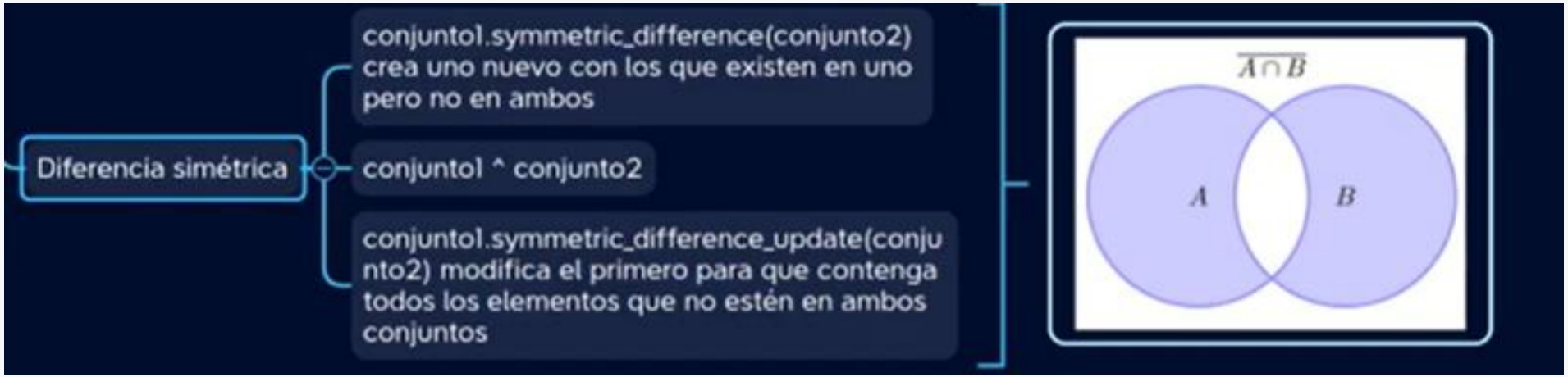
Conjuntos – Operaciones (II)





Conjuntos – Operaciones (III)





Diccionarios

Concepto:

- Los diccionarios son una estructura de datos versátil que permite almacenar pares de datos conocidos como <clave= y <valor=
- No permiten duplicados y son de muy rápido acceso. Velocidad de búsqueda constante.
- No poseen índices como las listas o arreglos.
- Simulan arreglos con índices que no son números



Métodos

- Creación. dict(). Y asignación de pares con llaves con la clave y valor separados con el signo “:” como indicador.

Diccionarios. Metodos

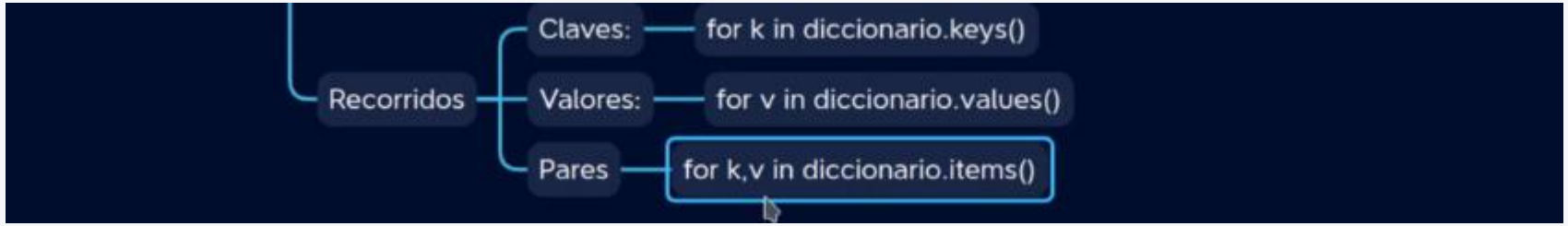
Métodos: (cont)

- Creación. dict(). Y asignación de pares con llaves con la clave y valor separados con el signo “:” como indicador.
- Búsqueda. Retorna el valor que posee la clave que se pasa entre corchetes [].Ej: diccionario[“ene”]
- Inserción y Modificación. diccionario[“ene”]= <valor>
- Borrado. del diccionario[<clave><default>] y diccionario.pop[<clave>] que retorna el valor.
- Recorrido. Retorna claves (keys), valores (values) y pares (items).

```python
diccionario = {1: "Lunes", 2: "Martes", 3: "Miércoles"}

for numero, nombre in diccionario.items():
    print(f"El día {numero} se llama {nombre}")
```

Diccionario – Operaciones



Operaciones:

- Unión
- Intersección
- Diferencia
- Diferencia Simétrica
- Todos se pueden aplicar a los resultados de keys, values, items.

CIERRE

Ideas clave de la clase

Usar esta diapositiva para consolidar los conceptos antes de finalizar.

1

1. Modelar antes de programar

El código refleja decisiones de análisis.

2

2. Diseñar responsabilidades

Las clases deben colaborar, no acumular lógica sin criterio.

3

3. Validar reglas de negocio

La consistencia no es opcional: forma parte del sistema.

Pregunta de salida

¿Qué decisión de diseño tomada hoy cambiarías si el sistema tuviera que crecer durante un año?

DAO

Próxima clase

Del modelo al código: clases, constructores y objetos en Python.

